Linear Data Structures Toolkit (LDST)

Unit–2 Assignment Report

Student Details

Name: Jahan Gaur

Roll No: 2501730484

Course: B.Tech CSE AI/ML

Section: G

---

1. Introduction

This project implements fundamental linear data structures in Python, including Dynamic Array, Linked Lists, Stack, and Queue. These structures are used to solve a real-world problem: checking balanced parentheses in expressions.

---

2. Dynamic Array

Concept

A dynamic array automatically resizes when it becomes full. Unlike static arrays, it can grow in size dynamically.

Operations Implemented

- append(x)
- pop()
- display()

Amortized Time Complexity of append()

Most append operations take **O(1)** time because elements are added at the end.

However, when the array becomes full:

- A new array of double capacity is created
- All elements are copied → takes **O(n)** time

Since resizing happens rarely, the **average time per append is O(1)**.

👉 This is called **amortized O(1)** time complexity.

---

3. Singly Linked List

Concept

A linked list stores elements as nodes connected via pointers.

Operations

- Insert at beginning → O(1)
- Insert at end → O(n)
- Delete by value → O(n)
- Traverse → O(n)

Advantage

No shifting required like arrays.

---

4. Doubly Linked List

Concept

Each node has:

- next pointer
- previous pointer

Operations

- Insert after node → O(n)
- Delete at position → O(n)

Advantage

Traversal possible in both directions.

---

5. Stack (LIFO)

Concept

Last In First Out structure.

Operations

- push → O(1)
- pop → O(1)
- peek → O(1)

Implementation

Using Singly Linked List (insertion/deletion at head).

---

6. Queue (FIFO)

Concept

First In First Out structure.

Operations

- enqueue → O(1)
- dequeue → O(1)
- front → O(1)

Implementation

Using head and tail pointers for efficiency.

---

7. Application: Balanced Parentheses Checker

Logic

- Push opening brackets onto stack
- On closing bracket:
    - Pop from stack
    - Check if it matches

Examples

- "([])" → Balanced
- "([)]" → Not Balanced
- "((" → Not Balanced
- "" → Balanced

---

8. Conclusion

This assignment helped in understanding how linear data structures work internally and how they can be applied to solve real-world problems efficiently.

---

9. References

- GeeksforGeeks
- Python Documentation
- Lecture Notes